

Database - No SQL- Mongodb

1. Explain why Node.js is preferred for real-time backend APIs.

Answer:

Node.js is preferred for real-time backend APIs because it uses an event-driven, non-blocking architecture. It can handle many client requests simultaneously without creating a separate thread for each request. This makes it fast and efficient for applications like chat apps, live notifications, online gaming, and real-time dashboards.

2. Explain non-blocking I/O using a file read example.

Answer:

Non-blocking I/O means Node.js does not wait for an operation to finish before executing the next task.

➤ Example:

```
const fs = require('fs');  
  
console.log("Start");  
  
fs.readFile('data.txt', 'utf8', (err, data) => {  
    console.log(data);  
});  
  
console.log("End");
```

➤ Output:

Start

End

(File content appears later)

➤ Explanation:

Node.js starts reading the file and immediately moves to the next statement instead of waiting for the file to be read. When the file reading is complete, the callback function executes.

3. Explain how Promises prevent callback hell.

Answer:

Callback hell occurs when multiple asynchronous operations are nested inside each other, making the code difficult to read and maintain.

➤ **Callback Hell Example:**

```
getUser(function(user) {  
  getOrders(user.id, function(orders) {  
    getPayment(orders[0].id, function(payment) {  
      console.log(payment);  
    });  
  });  
});
```

➤ **Using Promises:**

```
getUser()  
  .then(user => getOrders(user.id))  
  .then(orders => getPayment(orders[0].id))  
  .then(payment => console.log(payment))  
  .catch(err => console.log(err));
```

➤ **Benefits:**

- Cleaner code
- Better readability
- Easier error handling using .catch()
- Avoids deeply nested callbacks

4. Explain MongoDB document structure for product catalogs.

Answer:

MongoDB stores data in documents using JSON-like format called BSON. A product catalog can store product details in a single document.

➤ **Example Product Document:**

```
{  
  "_id": "101",  
  "name": "Laptop",  
  "brand": "Dell",  
  "price": 55000,  
  "category": "Electronics",  
  "stock": 25,  
  "rating": 4.5  
}
```

➤ **Advantages:**

- Flexible schema
- Easy to store complex product information
- Fast retrieval of product data
- Suitable for e-commerce applications

5. Explain why aggregation pipelines are essential for reports.

Answer:

Aggregation pipelines in MongoDB are used to process and analyze data for generating reports. They allow filtering, grouping, sorting, and calculating results from large datasets.

➤ **Example:**

```
db.orders.aggregate([  
  {  
    $group: {  
      _id: "$category",  
      totalSales: { $sum: "$amount" }  
    }  
  }  
]);
```

➤ **Use Cases:**

- Total sales report
- Monthly revenue report
- Top-selling products
- Customer purchase analysis

➤ **Benefits:**

- Performs calculations on the database server
 - Faster report generation
 - Reduces application-side processing
 - Handles large amounts of data efficiently
-

❖ Interview Short Answers

1) Why Node.js?

Node.js is fast, scalable, and uses non-blocking I/O, making it ideal for real-time applications.

2) What is Non-Blocking I/O?

It allows Node.js to continue executing other tasks while waiting for an operation to complete.

3) How do Promises prevent callback hell?

Promises use `.then()` and `.catch()` to chain asynchronous operations, making code cleaner and easier to manage.

4) MongoDB document structure?

MongoDB stores product data as flexible JSON-like documents containing product details such as name, price, category, and stock.

5) Why use aggregation pipelines?

They help generate reports by filtering, grouping, and calculating data efficiently within MongoDB.